p0593r1
Implicit creation of objects for low-level object manipulation

Draft Proposal, 16 October 2017

This version:
http://wg21.link/p0593r1

Author:
Richard Smith (Google)

Former Author:
Ville Voutilainen

Audience:
SG12

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

This paper proposes that objects of sufficiently trivial types be created on-demand as necessary to give programs defined behavior.

Table of Contents

1 Motivating examples
1.1 Idiomatic C code as C++
1.2 Objects provided as byte representation
1.3 Dynamic construction of arrays

2 Approach
2.1 Affected types
2.2 When to create objects
2.3 Type punning
2.4 Constant expressions
2.5 Practical examples

3 Acknowledgements

§ 1. Motivating examples

§ 1.1. Idiomatic C code as C++

Consider the following natural C program:

```
struct X { int a, b; };
X *make_x() {
  X *p = (X*)malloc(sizeof(struct X));
  p->a = 1;
  p->b = 2;
  return p;
}
```

When compiled with a C++ compiler, this code has undefined behavior, because `p->a` attempts to write to an `int` subobject of an `X` object, and this program never created either an `X` object nor an `int` subobject.

Per [intro.object]p1,

> An *object* is created by a definition, by a *new-expression*, when implicitly changing the active member of a union, or when a temporary object is created.

... and this program did none of these things.

1.2. Objects provided as byte representation

Suppose a C++ program is given a sequence of bytes (perhaps from disk or from a network), and it knows those bytes are a valid representation of type `T`. How can it efficiently obtain a `T *` that can be legitimately used to access the object?

Example: (many details omitted for brevity)

```
void process(Stream *stream) {
  unique_ptr<char[]> buffer = stream->read();
  if (buffer[0] == FOO)
    process_foo(reinterpret_cast<Foo*>(buffer.get())); // #1
  else
    process_bar(reinterpret_cast<Bar*>(buffer.get())); // #2
}
```

This code leads to undefined behavior today: within `Stream::read`, no `Foo` or `Bar` object is created, and so any attempt to access a `Foo` object through the `Foo*` produced by the cast at #1 would result in undefined behavior.

1.3. Dynamic construction of arrays

Consider this program that attempts to implement a type like `std::vector` (with many details omitted for brevity):

```
template<typename T> struct Vec {
  char *buf = nullptr, *buf_end_size = nullptr, *buf_end_capacity = nullptr;
  void reserve(std::size_t n) {
    char *newbuf = (char*)::operator new(n * sizeof(T), std::align_val_t(alignof(T)));
    std::uninitialized_copy(begin(), end(), (T*)newbuf); // #a

    ::operator delete(buf, std::align_val_t(alignof(T)));
    buf_end_size = newbuf + sizeof(T) * size(); // #b
    buf_end_capacity = newbuf + sizeof(T) * n;  // #c
    buf = newbuf;
  }
  void push_back(T t) {
    if (buf_end_size == buf_end_capacity)
      reserve(std::max<std::size_t>(size() * 2, 1));
    new (buf_end_size) T(t);
    buf_end_size += sizeof(T); // #d
  }
  T *begin() { return (T*)buf; }
  T *end() { return (T*)buf_end_size; }
  std::size_t size() { return end() - begin(); } // #e
};
int main() {
  Vec<int> v;
  v.push_back(1);
  v.push_back(2);
  v.push_back(3);
  for (int n : v) { /*...*/ } // #f
}
```

In practice, this code works across a range of existing implementations, but according to the C++ object model, undefined behavior occurs at points #a, #b, #c, #d, and #e, because they attempt to perform pointer arithmetic on a region of allocated storage that does not contain an array object.

At locations #b, #c, and #d, the arithmetic is performed on a `char*`, and at locations #a, #e, and #f, the arithmetic is performed on a `T*`. Ideally, a solution to this problem would imbue both calculations with defined behavior.

§ 2. Approach

The above snippets have a common theme: they attempt to use objects that they never created. Indeed, there is a family of types for which programmers assume they do not need to explicitly create objects. We propose to identify these types, and carefully carve out rules that remove the need to explicitly create such objects, by instead creating them implicitly.

§ 2.1. Affected types

If we are going to create objects automatically, we need a bare minimum of the following two properties for the type:

1) *Creating an instance of the type runs no code.* For class types, having a trivially default constructible type is likely the right constraint.

2) *Destroying an instance of the type runs no code.* If the type maintains invariants, we should not be implicitly creating objects of that type.

Note that we're only interested in properties of the object itself here, not of its subobjects. In particular, the above two properties always hold for array types. While creating or destroying array elements might run code, creating the array object (without its elements) does not.

This suggests that the largest set of types we could apply this to is:

- Scalar types

- Array types (with any element type)
- Class types with a trivial default constructor and a trivial destructor

(Put another way, we can apply this to all types other than function type, reference type, `void`, and class types with non-trivial default constructors or destructors.)

However, there are additional cases where we may wish to be conservative; if a class type has a non-trivial copy or move constructor, for instance, we should probably not permit objects to come into being without explicit user action. Therefore, we suggest permitting this for only trivial class types.

We will call types that satisfy the above constraints *implicit lifetime types*.

§ 2.2. When to create objects

In the above cases, it would be sufficient for `malloc` / `::operator new` to implicitly create sufficient objects to make the examples work. Imagine that `malloc` could "look into the future" and see how its storage would be used, and create the set of objects that the program would eventually need. If we somehow specified that `malloc` did this, the behavior of many C-style use cases would be defined.

On typical implementations, we can argue that this is not only natural, it is in some sense the status quo. Because the compiler typically does not make assumptions about what objects are created within the implementation of `malloc`, and because object creation itself typically has no effect on the physical machine, the compiler must generate code that would be correct if `malloc` did create that correct set of objects.

However, this is not always sufficient. An allocation from `malloc` may be sequentially used to store multiple different types, for instance by way of a memory pool that recycles the same allocation for multiple objects of the same size. Such uses should also have defined behavior.

Therefore we propose the following rule:

> The abstract machine creates objects of implicit lifetime types as needed to give the program defined behavior. If there exists a set of such objects, with corresponding points of creation, whose creation would give the program defined behavior, then the program has that behavior. Otherwise, the behavior of the program is undefined.

The coherence of the above rule hinges on a key observation: changing the set of objects that are implicitly created can only change whether a particular program execution has defined behavior, not what the behavior is.

The point of creation of such an implicit object is sequenced either immediately before or immediately after some evaluation within the program execution. For simplicity, we require such an evaluation to already exist in the program's execution. (This theoretically might not be the case if two unsequenced evaluations in distinct threads race to create a new object; we do not propose to cover such cases.)

§ 2.3. Type punning

We do not wish an example such as the following to become valid:

```
float do_bad_things(int n) {
  alignof(int) alignof(float)
    char buffer[max(sizeof(int), sizeof(float))];
  *(int*)buffer = n;       // #1
  return (*float*)buffer; // #2
}
```

The proposed rule would permit an `int` object to spring into existence to make line #1 valid, and would permit a `float` object to likewise spring into existence to make line #2 valid.

However, this example still does not have defined behavior under the proposed rule. The reason is a consequence of [basic.life]p4:

> The properties ascribed to objects and references throughout this document apply for a given object or reference only during its lifetime.

Specifically, the value held by an object is only stable throughout its lifetime. When the lifetime of the `int` object in line #1 ends (when its storage is reused by the `float` object in line #2), its value is gone. Symmetrically, when the `float` object is created, the object has an indeterminate value ([dcl.init]p12), and therefore any attempt to load its value results in undefined behavior.

Thus we retain the property (essential to modern scalar type-based alias analysis) that loads of some scalar type can be considered to not alias earlier stores of unrelated scalar types.

§ 2.4. Constant expressions

Constant expression evaluation is currently very conservative with regard to object creation. There is a tension here: on the one hand, constant expression evaluation gives us an opportunity to disallow runtime program semantics that we consider undesirable or problematic, and on the other hand, users strongly desire a full compile-time evaluation mechanism with the same semantics as the base language.

Following the existing conservatism in constant expression evaluation (eg, the disallowance of changing the active member of a union), we propose that the implicit creation of objects should *not* be performed during such evaluation.

§ 2.5. Practical examples

```
std::vector<int> vi;
vi.reserve(4);
vi.push_back(1);
int *p = &vi.back();
vi.push_back(2);
vi.push_back(3);
int n = *p;
```

Within the implementation of `vector`, some storage is allocated to hold an array of up to 5 `int`s. Ignoring minor differences, there are two ways to create implicit objects to give the execution of this program defined behavior: within the allocated storage, either an `int[3]` object or an `int[4]` object is created. Both are correct interpretations of the program, and naturally both result in the same behavior. We can choose to view the program as being in the superposition of those two states. If we add a fourth `push_back` call to the program prior to the initialization of `n`, then only the `int[4]` interpretation remains valid.

```
void process(Stream *stream) {
  unique_ptr<char[]> buffer = stream->read();
  if (buffer[0] == FOO)
    process_foo(reinterpret_cast<Foo*>(buffer.get())); // #1
  else
    process_bar(reinterpret_cast<Bar*>(buffer.get())); // #2
}
```

In this case, the program would have defined behavior if an object of type `Foo` or `Bar` (as appropriate for the content of the incoming data) were implicitly created *prior* to `Stream::read` populating its buffer. Therefore, regardless of which arm of the `if` is taken, there is a set of implicit objects sufficient to give the program defined behavior, and thus the behavior of the program is defined.

§ 3. Acknowledgements

Thanks to Ville Voutilainen for raising this problem, and to the members of SG12 for discussing possible solutions.